Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación

Licenciatura en Ciencias de la Computación / Ingeniería en Ciencias de la Computación /Ingeniería en Tecnologías de la Información / Ingeniería en Ciencia de Datos

FCCA08 Estructuras de Datos y sus Aplicaciones

EXAMEN DEPARTAMENTAL

"Editor de grafos"

Otoño, 2026.

1. Descripción general

Editor de Grafos es un proyecto de software educativo diseñado para permitir la creación, edición, visualización y análisis de grafos, con el propósito de representar gráficamente el funcionamiento de algoritmos fundamentales de la teoría de grafos y de las estructuras de datos.

La aplicación estará orientada principalmente al aprendizaje de los algoritmos de árbol de expansión mínima y caminos más cortos, permitiendo al usuario construir un grafo de manera visual, definir sus características y posteriormente ejecutar algoritmos que muestren gráficamente el proceso de solución.

El sistema permitirá trabajar con un máximo de 20 × 20 nodos, proporcionando una representación visual y una representación matricial del grafo mediante su matriz de adyacencia.

Los algoritmos principales que serán implementados son:

- Kruskal, para obtener el árbol de expansión mínima.
- Prim, para obtener el árbol de expansión mínima.
- Dijkstra, para determinar los caminos más cortos desde un nodo origen.
- Floyd-Warshall, para determinar los caminos más cortos entre todos los pares de nodos.

El propósito fundamental del proyecto es que el estudiante no solamente obtenga el resultado de un algoritmo, sino que pueda observar gráficamente cómo se construye la solución paso a paso.

---

2. Creación gráfica de los grafos

Una de las características principales del Editor de Grafos será la posibilidad de crear los nodos directamente sobre un área gráfica de trabajo.

El usuario podrá seleccionar una opción para agregar un nodo y posteriormente indicar mediante el cursor su posición dentro del área de edición. Cada nodo tendrá un identificador único, por ejemplo:

A, B, C, D, E...

A partir de los nodos creados, el usuario podrá establecer las conexiones entre ellos mediante aristas.

El sistema deberá permitir modificar o eliminar nodos y aristas, facilitando la construcción y edición del grafo antes de ejecutar cualquiera de los algoritmos.

La representación gráfica deberá mostrar claramente:

- Los nodos.
- Las aristas.
- La dirección de las aristas cuando corresponda.
- El peso asociado a cada arista.
- El nodo seleccionado como origen.
- El resultado obtenido por el algoritmo.

---

3. Grafos dirigidos y no dirigidos

El sistema permitirá determinar si el grafo que se está construyendo es:

- Grafo dirigido.
- Grafo no dirigido.

En un grafo no dirigido, una conexión entre los nodos A y B representa una relación en ambos sentidos:

A — B

Mientras que en un grafo dirigido, la conexión tendrá una dirección determinada:

A → B

Esta característica deberá establecerse antes o durante la construcción del grafo, y la representación gráfica deberá utilizar flechas para indicar la dirección de las aristas cuando se trate de un grafo dirigido.

El tipo de grafo tendrá relevancia en la ejecución de los algoritmos, ya que no todos los algoritmos están diseñados para trabajar bajo las mismas condiciones.

---

4. Pesos de las aristas

El Editor de Grafos permitirá indicar si las aristas tendrán peso.

Cuando el grafo sea ponderado, el usuario podrá asignar un valor numérico a cada arista. Este valor puede representar, por ejemplo:

- Distancia.
- Tiempo.
- Costo.
- Consumo.
- Latencia.
- O cualquier otra magnitud que pueda modelarse mediante un valor numérico.

Por ejemplo:

A — 5 — B

representará una conexión entre A y B cuyo peso es 5.

Esta característica resulta indispensable para los algoritmos de Kruskal, Prim y Dijkstra, ya que estos algoritmos utilizan los pesos de las aristas para determinar sus soluciones.

En el caso de Floyd-Warshall, los pesos también serán utilizados para calcular las distancias mínimas entre los diferentes pares de nodos.

El sistema deberá validar que los pesos introducidos cumplan las condiciones requeridas por el algoritmo que se desea ejecutar.

---

5. Construcción del grafo mediante matriz de adyacencia

Además de permitir la construcción gráfica, el sistema proporcionará una segunda alternativa: crear o modificar el grafo mediante su matriz de adyacencia.

La matriz de adyacencia permitirá representar las relaciones existentes entre los nodos mediante una estructura bidimensional de hasta 20 × 20 posiciones.

Por ejemplo, para un grafo de cuatro nodos:

	A	B	C	D
A	0	4	2	∞
B	4	0	1	5
C	2	1	0	8
D	∞	5	8	0

La matriz permitirá establecer o modificar las conexiones y sus pesos.

Una característica importante del proyecto será la sincronización entre ambas representaciones. Es decir:

Matriz de adyacencia → Representación gráfica

y también:

Representación gráfica → Matriz de adyacencia

De esta manera, cuando el usuario modifique una arista gráficamente, la matriz deberá actualizarse, y cuando modifique un valor de la matriz, la representación gráfica deberá reflejar el cambio.

---

6. Representación visual de los algoritmos

Una característica fundamental del proyecto será la capacidad de visualizar el funcionamiento de los algoritmos.

El sistema no deberá limitarse a mostrar el resultado final. Deberá permitir que el usuario observe, preferentemente paso a paso, las decisiones tomadas durante la ejecución.

Kruskal

El algoritmo de Kruskal deberá mostrar cómo se seleccionan progresivamente las aristas de menor peso para construir el árbol de expansión mínima.

Las aristas seleccionadas podrán destacarse visualmente, mientras que aquellas descartadas debido a que formarían un ciclo podrán identificarse de manera diferente.

Prim

El algoritmo de Prim deberá mostrar cómo se construye progresivamente el árbol de expansión mínima a partir de un nodo inicial.

El usuario podrá observar cómo se incorporan nuevos nodos y aristas al árbol.

Dijkstra

Dijkstra deberá permitir seleccionar un nodo origen y mostrar progresivamente la actualización de las distancias mínimas hacia los demás nodos.

El sistema podrá mostrar:

- Nodo actual.
- Distancia acumulada.
- Nodos visitados.
- Aristas consideradas.
- Distancias actualizadas.
- Camino mínimo encontrado.

Floyd-Warshall

Floyd-Warshall deberá mostrar el proceso de actualización de la matriz de distancias.

El usuario podrá observar cómo se consideran progresivamente los diferentes nodos como intermediarios para determinar los caminos mínimos entre todos los pares de nodos.

---

7. Arquitectura funcional propuesta

El proyecto puede organizarse en los siguientes módulos:

Módulo de edición

Permitirá:

- Crear nodos.
- Mover nodos.
- Eliminar nodos.
- Crear aristas.
- Eliminar aristas.
- Modificar pesos.
- Definir grafos dirigidos o no dirigidos.

Módulo de matriz

Permitirá:

- Visualizar la matriz de adyacencia.
- Editar sus valores.
- Crear el grafo a partir de la matriz.
- Actualizar la matriz a partir del grafo.

Módulo de algoritmos

Contendrá las implementaciones de:

- Kruskal.
- Prim.

- Dijkstra.
- Floyd-Warshall.

Módulo de visualización

Será responsable de representar gráficamente:

- El grafo original.
- Las aristas seleccionadas.
- Los nodos visitados.
- Los caminos encontrados.
- Los árboles de expansión mínima.
- Las matrices resultantes.

Módulo de control de ejecución

Permitirá ejecutar los algoritmos:

- Automáticamente.
- Paso a paso.
- Reiniciando la ejecución.
- Avanzando o retrocediendo entre etapas, si esta funcionalidad se incorpora al proyecto.

---

8. Restricción del tamaño del grafo

Para mantener una representación gráfica clara y facilitar el uso educativo de la aplicación, el sistema tendrá como límite máximo:

20 × 20 nodos.

Esto significa que el editor podrá trabajar con grafos de hasta 20 nodos, y la matriz de adyacencia tendrá como máximo 20 filas × 20 columnas.

El sistema deberá impedir que el usuario agregue nodos cuando se haya alcanzado el límite establecido.

---

9. Objetivo educativo

El objetivo principal del Editor de Grafos será proporcionar una herramienta que permita relacionar los conceptos teóricos de estructuras de datos, grafos y algoritmos con una representación visual e interactiva.

El estudiante podrá construir un problema, modificar sus características, observar su representación mediante una matriz de adyacencia y ejecutar diferentes algoritmos sobre el mismo grafo.

De esta manera, el proyecto permitirá comprender experimentalmente conceptos como:

- Grafos dirigidos y no dirigidos.
- Grafos ponderados.
- Matrices de adyacencia.
- Árboles de expansión mínima.
- Caminos mínimos.
- Recorridos y selección de aristas.
- Complejidad algorítmica.
- Comparación entre diferentes algoritmos.

En términos generales, Editor de Grafos será una herramienta educativa que integrará en una misma aplicación la construcción gráfica, representación matricial y ejecución visual de algoritmos, permitiendo que el estudiante comprenda no solamente cuál es la solución obtenida, sino también cómo el algoritmo llega a dicha solución.

Consideraciones generales: Los estudiantes deberán organizarse en parejas colaborativas (máximo 2 personas) y subir las evidencias solicitadas en la tarea asignada en Teams en el período establecido (ambas cuentas de la pareja colaborativa).

Rúbrica de evaluación

Objetivo de la evaluación

Evaluar el desarrollo de una aplicación de software capaz de crear, editar, representar y analizar grafos, incorporando los algoritmos de Kruskal, Prim, Dijkstra y Floyd-Warshall, tanto desde una representación gráfica como mediante una matriz de adyacencia.

La evaluación considera aspectos de funcionalidad, implementación algorítmica, representación gráfica, interacción, validación, calidad del código y documentación.

---

2. Rúbrica general

Criterio	Excelente	Bueno	Satisfactorio	Insuficiente	Puntos
1. Creación y edición de grafos	Permite crear, modificar, mover y eliminar nodos y aristas de	La mayoría de las funciones están implementadas correctamente.	Permite crear grafos, pero presenta algunas limitaciones.	La creación o edición presenta fallas importantes.	10

	forma completa y funcional.				
2. Grafos dirigidos y no dirigidos	Permite seleccionar y representar correctamente ambos tipos, incluyendo la dirección mediante flechas.	Implementa ambos tipos con pequeños errores.	Solo uno funciona completamente o existen limitaciones.	No diferencia correctamente entre ambos tipos.	8
3. Grafos ponderados	Permite asignar, modificar y visualizar correctamente los pesos de las aristas.	Funciona correctamente con pequeños problemas de interacción.	Permite pesos, pero con restricciones o errores ocasionales.	No implementa adecuadamente los pesos.	8
4. Matriz de adyacencia	Permite visualizar, editar y generar el grafo desde una matriz de hasta 20×20.	Implementa la matriz con pequeñas limitaciones.	La matriz funciona parcialmente.	No existe una integración funcional con la matriz.	12
5. Sincronización gráfico ↔ matriz	Los cambios realizados en cualquiera de las dos representaciones se reflejan correctamente en la otra.	La sincronización funciona en la mayoría de los casos.	La sincronización es parcial.	Las representaciones son independientes o presentan errores graves.	8
6. Algoritmo de Kruskal	Implementación correcta y visualización paso a paso del árbol de expansión mínima.	Implementación correcta con pequeños errores visuales o de interacción.	Obtiene algunos resultados correctos, pero presenta errores en determinados casos.	No obtiene correctamente el árbol de expansión mínima.	10
7. Algoritmo de Prim	Implementación correcta y visualización clara del proceso de construcción del árbol.	Funciona correctamente con pequeñas limitaciones.	Funciona parcialmente o presenta errores en algunos casos.	No implementa correctamente el algoritmo.	10
8. Algoritmo de Dijkstra	Calcula correctamente los caminos	Calcula correctamente los caminos	Funciona parcialmente o presenta	No calcula correctamente	10

	mínimos y muestra claramente el proceso desde el nodo origen.	con pequeñas limitaciones visuales.	errores en algunos casos.	los caminos mínimos.	
9. Algoritmo de Floyd-Warshall	Calcula correctamente los caminos mínimos entre todos los pares y muestra el proceso mediante la matriz.	Funciona correctamente con pequeñas limitaciones.	Presenta resultados parciales o errores en determinados casos.	No implementa correctamente el algoritmo.	10
10. Visualización de resultados	El proceso de cada algoritmo se muestra claramente, diferenciando nodos, aristas, caminos y resultados.	La visualización es clara, aunque puede mejorar.	La visualización permite identificar el resultado, pero es limitada.	Los resultados son difíciles de interpretar.	5
11. Validaciones y manejo de errores	Detecta adecuadamente entradas inválidas, grafos incompatibles, pesos incorrectos y límite de 20 nodos.	Cuenta con la mayoría de las validaciones necesarias.	Tiene algunas validaciones básicas.	Carece de validaciones importantes.	4
12. Calidad del código y documentación	Código organizado, modular, legible, comentado y acompañado de documentación técnica y manual de usuario.	Código organizado y documentación suficiente.	Código funcional pero con organización o documentación limitada.	Código desorganizado y sin documentación suficiente.	5
TOTAL					100